

新しい強化学習環境のデータセットを追加するためのガイド

このドキュメントでは、PLDMリポジトリに新しい強化学習環境のデータセットを追加する際に必要な変更と新規ファイルについて説明します。

概要

新しい環境（例: MyEnv）を追加する場合、以下の手順が必要です:

1. 環境固有のデータセットクラスを実装
2. データ設定（Config）を定義
3. **DatasetType**に新しいタイプを追加
4. **DataConfig**に新しい設定を追加
5. **DatasetFactory**に作成ロジックを追加
6. （オプション）評価・プランニングロジックを追加

前提知識

既存の環境構成

現在のリポジトリには以下の環境が実装されています:

```

pldm_envs/
├── wall/                # 壁のある2D環境
│   ├── data/           # データセット実装
│   ├── evaluation/     # 評価用環境
│   └── configs/        # YAML設定ファイル
├── diverse_maze/       # D4RLベンチマーク（迷路環境）
│   ├── data_generation/ # データ生成スクリプト
│   ├── evaluation/     # 評価用環境
│   └── configs/        # YAML設定ファイル
└── utils/              # 共通ユーティリティ（Normalizerなど）

```

ステップ1: 環境ディレクトリの作成

1.1 ディレクトリ構造

新しい環境用のディレクトリを作成します:

```

mkdir -p pldm_envs/myenv/{data,evaluation,configs}
touch pldm_envs/myenv/__init__.py

```

推奨ディレクトリ構造:

```

pldm_envs/myenv/
├── __init__.py
├── README.md           # 環境の説明
├── data/
│   ├── __init__.py
│   ├── myenv_dataset.py # メインデータセットクラス
│   └── utils.py         # データセット用ユーティリティ
├── evaluation/
│   ├── __init__.py
│   └── envs_generator.py # 評価用環境生成
├── configs/
│   └── default.yaml     # デフォルト設定
└── enums.py             # データセット設定とサンプル定義

```

ステップ2: データセットクラスの実装

2.1 サンプルデータ構造の定義

ファイル: `p1dm_envs/myenv/enums.py`

```

from typing import NamedTuple, Optional
import torch
from dataclasses import dataclass
from omegaconf import MISSING

# データセットが返すサンプルの構造を定義
class MyEnvSample(NamedTuple):
    """
    データセットの1サンプルを表すNamedTuple

    必須フィールド:
    - states: 観測 (画像またはベクトル)
    - actions: アクション系列

    オプションフィールド (環境に応じて追加):
    - locations: エージェントの位置座標
    - propio_pos: 固有受容位置 (ロボットの関節角度など)
    - propio_vel: 固有受容速度 (関節速度など)
    - goal: ゴール情報
    """
    states: torch.Tensor          # [T, C, H, W] or [T, D]
    actions: torch.Tensor         # [T-1, Action_Dim]
    locations: Optional[torch.Tensor] = None # [T, Pos_Dim]
    propio_pos: Optional[torch.Tensor] = None # [T, Propio_Dim]
    propio_vel: Optional[torch.Tensor] = None # [T, Propio_Dim]
    goal: Optional[torch.Tensor] = None      # [Goal_Dim]

@dataclass
class MyEnvDatasetConfig:
    """
    データセットの設定

    重要なパラメータ:
    - env_name: 環境名 (評価時に使用)
    - batch_size: バッチサイズ
    - sample_length: サンプルのタイムステップ数
    - train: 訓練/検証モードの切り替え
    - image_based: 画像ベースか固有受容情報ベースか
    """
    env_name: str = MISSING
    batch_size: int = 128
    sample_length: int = 17          # タイムステップ数

```

```
train: bool = True

# データソース
data_path: Optional[str] = None
images_path: Optional[str] = None

# データ設定
image_based: bool = True
img_size: int = 64
stack_states: int = 1          # 状態のスタック数

# データセット管理
num_workers: int = 0
crop_length: Optional[int] = None # データセット長の制限
quick_debug: bool = False

# 環境固有のパラメータ
# 例: 報酬の閾値、特定の初期化パラメータなど
# custom_param: float = 1.0
```

2.2 データセットクラスの実装

ファイル: `pldm_envs/myenv/data/myenv_dataset.py`

```

import torch
import numpy as np
from typing import Optional
from pldm_envs.myenv.enums import MyEnvSample, MyEnvDatasetConfig

class MyEnvDataset(torch.utils.data.Dataset):
    """
    新しい環境のデータセットクラス

    実装すべきメソッド:
    - __init__: データのロードと前処理
    - __len__: データセットの長さを返す
    - __getitem__: インデックスに対応するサンプルを返す
    """

    def __init__(
        self,
        config: MyEnvDatasetConfig,
        normalizer=None, # pldm_envs.utils.normalizer.Normalizer
    ):
        self.config = config
        self.normalizer = normalizer # DataLoaderで設定される

        # データのロード
        self._load_data()

        print(f"Loaded MyEnv dataset with {len(self)} samples")

    def _load_data(self):
        """
        データをロードするメソッド

        実装例:
        1. NPZ/NPYファイルからロード
        2. HDF5ファイルからロード
        3. オンザフライ生成 (シミュレーション)
        """
        if self.config.data_path is not None:
            # 保存済みデータからロード
            data = np.load(self.config.data_path, allow_pickle=True)

            # 軌跡データの構造に応じて処理

```

```

# 例: episodeごとに分割されている場合
self.observations = data['observations'] # [N_episodes, T, ...]
self.actions = data['actions']          # [N_episodes, T-1, ...]

if 'locations' in data:
    self.locations = data['locations']
else:
    self.locations = None

else:
    # オンザフライでデータ生成 (シミュレーション環境の場合)
    raise NotImplementedError(
        "On-the-fly data generation not implemented. "
        "Please provide data_path."
    )

# 画像データのロード (別ファイルの場合)
if self.config.images_path is not None:
    if self.config.images_path.endswith('.npy'):
        # mmap_mode='r'でメモリ効率を向上
        self.images = np.load(
            self.config.images_path,
            mmap_mode='r' if not self.config.quick_debug else None
        )
    elif self.config.images_path.endswith('.zarr'):
        import zarr
        self.images = zarr.open(self.config.images_path, 'r')[:]

def __len__(self):
    """
    データセットの長さを返す

    注意:
    - crop_lengthが指定されている場合はそれを考慮
    - スライディングウィンドウを使う場合は適切に計算
    """
    if self.config.crop_length is not None:
        return min(self.config.crop_length, len(self.observations))
    return len(self.observations)

def __getitem__(self, idx: int) -> MyEnvSample:
    """
    インデックスに対応するサンプルを返す

```

Args:

idx: サンプルのインデックス

Returns:

MyEnvSample: 観測、アクション、その他の情報を含む

.....

データの取得

```
obs = self.observations[idx] # [T, ...]
```

```
actions = self.actions[idx] # [T-1, ...]
```

Tensorに変換

```
states = torch.from_numpy(obs).float()
```

```
actions = torch.from_numpy(actions).float()
```

画像の場合の処理

```
if self.config.image_based:
```

```
    if self.config.images_path is not None:
```

```
        # 別ファイルから画像をロード
```

```
        states = torch.from_numpy(self.images[idx]).float()
```

```
    # 正規化 (0-255 -> 0-1)
```

```
    if states.max() > 1:
```

```
        states = states / 255.0
```

位置情報の取得 (存在する場合)

```
if self.locations is not None:
```

```
    locations = torch.from_numpy(self.locations[idx]).float()
```

```
else:
```

```
    locations = None
```

サンプルを作成して返す

```
sample = MyEnvSample(
```

```
    states=states,
```

```
    actions=actions,
```

```
    locations=locations,
```

```
    propio_pos=None, # 必要に応じて実装
```

```
    propio_vel=None, # 必要に応じて実装
```

```
    goal=None,      # 必要に応じて実装
```

```
)
```

```
return sample
```



```

class MyEnvOnlineDataset:
    """
    オンライン生成データセット（シミュレーションからリアルタイムで生成）

    WallDatasetのようにバッチをまとめて生成する場合に使用
    """

    def __init__(self, config: MyEnvDatasetConfig):
        self.config = config
        # シミュレーション環境の初期化など

    def __len__(self):
        return self.config.size // self.config.batch_size

    def __iter__(self):
        """
        イテレーション時にバッチを生成

        注意：この場合はDataLoaderを使わず、
        make_data_loader_for_prebatched_ds()を使用
        """
        for _ in range(len(self)):
            yield self._generate_batch()

    def _generate_batch(self) -> MyEnvSample:
        """バッチを生成するロジック"""
        # シミュレーションを実行してデータを生成
        pass

```

2.3 データセット作成のベストプラクティス

メモリ効率:

```

# 大容量データの場合はmmapを使用
data = np.load(path, mmap_mode='r')

# zarr形式もメモリ効率が良い
import zarr
data = zarr.open(path, 'r')

```

データ検証:

```

def __init__(self, config):
    # ... データロード ...

    # データの形状を検証
    assert self.observations.shape[1] >= self.config.sample_length, \
        f"Episode length {self.observations.shape[1]} < sample_length {self.config.samp

    # アクション数の検証
    assert self.actions.shape[1] == self.observations.shape[1] - 1, \
        "Actions should have length T-1"

```

ステップ3: DatasetTypeとDataConfigへの追加

3.1 DatasetTypeに新しいタイプを追加

ファイル: [pldm/data/enums.py](#)

```

# 既存のコード（14-21行目）に追加
class DatasetType(Enum):
    Single = auto()
    Multiple = auto()
    Wall = auto()
    WallExpert = auto()
    D4RL = auto()
    D4RLEigf = auto()
    LocoMaze = auto()
    MyEnv = auto() # ← 追加

```

3.2 DataConfigに設定を追加

ファイル: [pldm/data/enums.py](#)

```

# 既存のコード（37-52行目）を編集
from pldm_envs.myenv.enums import MyEnvDatasetConfig # ← import追加

@dataclass
class DataConfig(ConfigBase):
    dataset_type: DatasetType = DatasetType.Single
    dot_config: DotDatasetConfig = DotDatasetConfig()
    wall_config: WallDatasetConfig = WallDatasetConfig()
    offline_wall_config: OfflineWallDatasetConfig = OfflineWallDatasetConfig()
    wall_expert_config: WallExpertDatasetConfig = WallExpertDatasetConfig()
    d4rl_config: D4RLDatasetConfig = D4RLDatasetConfig()
    myenv_config: MyEnvDatasetConfig = MyEnvDatasetConfig() # ← 追加

    normalize: bool = False
    min_max_normalize_state: bool = False
    normalizer_hardset: bool = False
    quick_debug: bool = False
    num_workers: int = 0

```

ステップ4: DatasetFactoryに作成ロジックを追加

4.1 create_datasets()に分岐を追加

ファイル: [pldm/data/dataset_factory.py](#)

```

# 既存のコード（1-2行目）にimport追加
import dataclasses

from pldm_envs.myenv.data.myenv_dataset import MyEnvDataset # ← 追加
from pldm_envs.wall.data.offline_wall import OfflineWallDataset
# ... 他のimport ...

```

既存のコード（34–46行目）を編集

```
def create_datasets(self):
    if self.config.dataset_type == DatasetType.Single:
        return self._create_single_datasets()
    elif self.config.dataset_type == DatasetType.Wall:
        return self._create_wall_datasets()
    elif self.config.dataset_type == DatasetType.WallExpert:
        return self._create_wall_expert_datasets()
    elif self.config.dataset_type == DatasetType.D4RL:
        return self._create_d4rl_datasets()
    elif self.config.dataset_type == DatasetType.LocoMaze:
        return self._create_locomaze_datasets()
    elif self.config.dataset_type == DatasetType.MyEnv: # ← 追加
        return self._create_myenv_datasets()           # ← 追加
    else:
        raise NotImplementedError
```

4.2 データセット作成メソッドの実装

ファイル: [pldm/data/dataset_factory.py](#)

ファイルの末尾に追加

```
def _create_myenv_datasets(self):
    """
    MyEnvデータセットを作成

    Returns:
        Datasets: 訓練データセット、検証データセット、プロービングデータセット
    """
    # 訓練データセット
    ds = MyEnvDataset(self.config.myenv_config)
    ds = make_data_loader(
        ds=ds,
        loader_config=self.config,
        suffix="myenv_train"
    )

    # 検証データセット (オプション)
    val_ds = None
    if self.config.myenv_config.val_path is not None:
        val_ds = MyEnvDataset(
            dataclasses.replace(
                self.config.myenv_config,
                data_path=self.config.myenv_config.val_path,
                images_path=self.config.myenv_config.val_images_path,
                train=False,
            )
        )
    val_ds = make_data_loader(
        ds=val_ds,
        loader_config=self.config,
        normalizer=ds.normalizer, # 訓練データのnormalizerを使用
        suffix="myenv_val",
        train=False,
    )

    # プロービング用データセット (オプション)
    probing_datasets = None
    if self.probing_cfg.train_path is not None:
        probing_datasets = self._create_myenv_probing_datasets(ds.normalizer)

    datasets = Datasets(
        ds=ds,
```

```
        val_ds=val_ds,  
        probing_datasets=probing_datasets,  
    )
```

```
    return datasets
```

```
def _create_myenv_probing_datasets(self, normalizer):  
    """
```

```
    プロービング評価用のデータセットを作成
```

```
    Args:
```

```
        normalizer: 訓練データから計算したnormalizer
```

```
    Returns:
```

```
        ProbingDatasets: プロービング用データセット
```

```
    """
```

```
    # プロービング訓練用
```

```
    probe_ds = MyEnvDataset(  
        dataclasses.replace(  
            self.config.myenv_config,  
            data_path=self.probing_cfg.train_path,  
            images_path=self.probing_cfg.train_images_path,  
            sample_length=self.probing_cfg.l1_depth,  
        )  
    )
```

```
    probe_ds = make_data_loader(  
        ds=probe_ds,  
        loader_config=self.config,  
        normalizer=normalizer,  
        suffix="myenv_probe_train",  
    )
```

```
    # プロービング検証用
```

```
    probe_val_ds = MyEnvDataset(  
        dataclasses.replace(  
            self.config.myenv_config,  
            data_path=self.probing_cfg.val_path,  
            images_path=self.probing_cfg.val_images_path,  
            sample_length=self.probing_cfg.l1_depth,  
            train=False,  
        )  
    )
```

```
probe_val_ds = make_data_loader(  
    ds=probe_val_ds,  
    loader_config=self.config,  
    normalizer=normalizer,  
    suffix="myenv_probe_val",  
)  
  
probing_datasets = ProbingDatasets(  
    ds=probe_ds,  
    val_ds=probe_val_ds,  
)  
  
return probing_datasets
```

ステップ5: 評価・プランニングロジックの追加（オプション）

表現学習の質を評価するために、プランニングやプロービングを実装する場合に必要です。

5.1 評価用環境の作成

ファイル: `pldm_envs/myenv/evaluation/envs_generator.py`

```

import numpy as np
from typing import List

class MyEnvGenerator:
    """
    評価用の環境を生成するクラス

    プランニング評価で使用する
    """

    def __init__(self, env_name: str, n_envs: int = 10):
        self.env_name = env_name
        self.n_envs = n_envs

    def generate_envs(self) -> List:
        """
        複数の評価用環境を生成

        Returns:
            List: 環境のリスト（初期状態とゴールを含む）
        """
        envs = []
        for i in range(self.n_envs):
            env_config = {
                'initial_state': self._generate_initial_state(),
                'goal': self._generate_goal(),
                'seed': i,
            }
            envs.append(env_config)
        return envs

    def _generate_initial_state(self):
        """初期状態を生成"""
        # 環境に応じた初期状態の生成
        pass

    def _generate_goal(self):
        """ゴールを生成"""
        # 環境に応じたゴールの生成
        pass

```


5.2 PixelMapperの実装（座標変換が必要な場合）

ファイル: `pldm_envs/myenv/utils.py`

```
import numpy as np
```

```
class MyEnvPixelMapper:
```

```
    """
```

```
    観測座標とピクセル座標の変換を行うクラス
```

```
    プロロービング評価で使用する
```

```
    """
```

```
    def __init__(self, env_name: str):
```

```
        self.env_name = env_name
```

```
        # 環境固有のパラメータを設定
```

```
        self.img_size = 64
```

```
        self.world_size = 10.0 # 実際の世界座標の範囲
```

```
    def obs_coord_to_pixel_coord(self, obs_coord: np.ndarray) -> np.ndarray:
```

```
        """
```

```
        観測座標をピクセル座標に変換
```

```
        Args:
```

```
            obs_coord: 観測座標 [..., 2]
```

```
        Returns:
```

```
            pixel_coord: ピクセル座標 [..., 2]
```

```
        """
```

```
        # 例: [-world_size/2, world_size/2] -> [0, img_size]
```

```
        pixel_coord = (obs_coord + self.world_size / 2) / self.world_size * self.img_si
```

```
        return pixel_coord
```

```
    def pixel_coord_to_obs_coord(self, pixel_coord: np.ndarray) -> np.ndarray:
```

```
        """
```

```
        ピクセル座標を観測座標に変換
```

```
        Args:
```

```
            pixel_coord: ピクセル座標 [..., 2]
```

```
        Returns:
```

```
            obs_coord: 観測座標 [..., 2]
```

```
        """
```

```
        obs_coord = pixel_coord / self.img_size * self.world_size - self.world_size / 2
```

```
        return obs_coord
```

5.3 Evaluatorの拡張

ファイル: [pldm/evaluation/evaluator.py](#)

```
# _create_pixel_mapper()メソッドを編集 (107-121行目)
def _create_pixel_mapper(self):
    if "diverse" in self.config.env_name or "maze2d" in self.config.env_name:
        pixel_mapper = D4RLPixelMapper(env_name=self.config.env_name)
    elif "myenv" in self.config.env_name: # ← 追加
        from pldm_envs.myenv.utils import MyEnvPixelMapper
        pixel_mapper = MyEnvPixelMapper(env_name=self.config.env_name)
    else:
        class IdPixelMapper:
            def obs_coord_to_pixel_coord(self, x):
                return x
            def pixel_coord_to_obs_coord(self, x):
                return x
        pixel_mapper = IdPixelMapper()

    return pixel_mapper

# _get_planning_config()メソッドを編集 (71-78行目)
def _get_planning_config(self):
    if "diverse" in self.config.env_name or "maze" in self.config.env_name:
        config = self.config.d4rl_planning
    elif self.config.env_name == "wall":
        config = self.config.wall_planning
    elif self.config.env_name == "myenv": # ← 追加
        config = self.config.myenv_planning # ← 追加 (EvalConfigで定義)
    else:
        raise NotImplementedError
    return config
```

5.4 EvalConfigへのプランニング設定追加 (必要な場合)

ファイル: [pldm/evaluation/evaluator.py](#)

```

# import文を追加
from pldm.planning.myenv.enums import MyEnvMPCCConfig

# EvalConfigクラスを編集 (20-34行目)
@dataclass
class EvalConfig(ConfigBase):
    env_name: str = MISSING
    probing: ProbingConfig = ProbingConfig()
    eval_l1: bool = True
    eval_l2: bool = False
    log_heatmap: bool = True
    disable_planning: bool = False
    wall_planning: WallMPCCConfig = WallMPCCConfig()
    d4rl_planning: D4RLMPCCConfig = D4RLMPCCConfig()
    myenv_planning: MyEnvMPCCConfig = MyEnvMPCCConfig() # ← 追加

    def __post_init__(self):
        self.wall_planning.env_name = self.env_name
        self.d4rl_planning.env_name = self.env_name
        self.myenv_planning.env_name = self.env_name # ← 追加

```

ステップ6: 設定ファイルの作成

6.1 YAML設定ファイル

ファイル: pldm_envs/myenv/configs/default.yaml

```
# MyEnv環境のデフォルト設定
```

```
env_name: myenv
```

```
# データ設定
```

```
data:
```

```
    dataset_type: MyEnv
```

```
    normalize: true
```

```
    min_max_normalize_state: false
```

```
    num_workers: 4
```

```
myenv_config:
```

```
    env_name: myenv
```

```
    batch_size: 128
```

```
    sample_length: 17
```

```
    train: true
```

```
# データパス
```

```
data_path: /path/to/myenv_train.npz
```

```
images_path: /path/to/myenv_images.npy
```

```
val_path: /path/to/myenv_val.npz
```

```
# データ設定
```

```
image_based: true
```

```
img_size: 64
```

```
stack_states: 1
```

```
# デバッグ
```

```
quick_debug: false
```

```
crop_length: null
```

```
# モデル設定
```

```
hjepa:
```

```
    level1:
```

```
        backbone:
```

```
            arch: resnet18 # or menet5
```

```
        # その他のモデルパラメータ
```

```
# 訓練設定
```

```
n_steps: 17
```

```
epochs: 100
```

```
base_lr: 0.2
```

```
# 評価設定
eval_cfg:
  env_name: myenv

  probing:
    l1_depth: 17
    probe_preds: true
    probe_encoder: true
    train_path: /path/to/myenv_probe_train.npz
    val_path: /path/to/myenv_probe_val.npz
```

6.2 実行コマンド

```
# 設定ファイルを使って訓練
python pldm/train.py --configs pldm_envs/myenv/configs/default.yaml

# コマンドラインから設定を上書き
python pldm/train.py \
  --configs pldm_envs/myenv/configs/default.yaml \
  --values \
    data.myenv_config.batch_size=256 \
    epochs=50
```

ステップ7: データの準備

7.1 データ生成スクリプト（オプション）

ファイル: `pldm_envs/myenv/data_generation/generate_data.py`

```
"""
```

MyEnv環境のデータを生成するスクリプト

```
"""
```

```
import numpy as np
```

```
import argparse
```

```
from tqdm import tqdm
```

```
def generate_episode(env, policy, max_steps=100):
```

```
    """
```

1エピソードのデータを生成

Args:

env: 環境

policy: 方策

max_steps: 最大ステップ数

Returns:

observations, actions, locations

```
    """
```

```
obs_list = []
```

```
action_list = []
```

```
location_list = []
```

```
obs = env.reset()
```

```
for _ in range(max_steps):
```

```
    obs_list.append(obs)
```

```
    location_list.append(env.get_location())
```

```
    action = policy(obs)
```

```
    obs, reward, done, info = env.step(action)
```

```
    action_list.append(action)
```

```
    if done:
```

```
        break
```

```
# 最後の観測を追加
```

```
obs_list.append(obs)
```

```
location_list.append(env.get_location())
```

```
return np.array(obs_list), np.array(action_list), np.array(location_list)
```

```

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--n_episodes', type=int, default=1000)
    parser.add_argument('--output_path', type=str, required=True)
    parser.add_argument('--policy_type', type=str, default='random')
    args = parser.parse_args()

    # 環境とポリシーの初期化
    # env = MyEnv()
    # policy = create_policy(args.policy_type)

    all_observations = []
    all_actions = []
    all_locations = []

    for i in tqdm(range(args.n_episodes), desc="Generating episodes"):
        obs, actions, locations = generate_episode(env, policy)
        all_observations.append(obs)
        all_actions.append(actions)
        all_locations.append(locations)

    # データを保存
    np.savez(
        args.output_path,
        observations=np.array(all_observations),
        actions=np.array(all_actions),
        locations=np.array(all_locations),
        terminals=np.zeros(len(all_observations)), # 終端フラグ
    )

    print(f"Saved {len(all_observations)} episodes to {args.output_path}")

if __name__ == '__main__':
    main()

```

7.2 データフォーマット

推奨されるデータフォーマット:

NPZ形式:


```
{
    'observations': np.ndarray, # [N_episodes, T, ...] or [N_episodes, T, C, H, W]
    'actions': np.ndarray,      # [N_episodes, T-1, Action_Dim]
    'locations': np.ndarray,     # [N_episodes, T, Pos_Dim] (オプション)
    'terminals': np.ndarray,    # [N_episodes, T] (オプション)
}
```

分離形式（大容量画像データの場合）：

- `myenv_trajectories.npz` : 軌跡メタデータ（アクション、位置など）
- `myenv_images.npy` : 画像データ（mmap対応）

ステップ8: テストとデバッグ

8.1 データセットの動作確認

ファイル: `pldm_envs/myenv/test_dataset.py`

```
"""
```

データセットの動作をテストするスクリプト

```
"""
```

```
from pldm_envs.myenv.enums import MyEnvDatasetConfig
from pldm_envs.myenv.data.myenv_dataset import MyEnvDataset
import matplotlib.pyplot as plt

def test_dataset():
    # 設定を作成
    config = MyEnvDatasetConfig(
        data_path='path/to/test_data.npz',
        batch_size=32,
        sample_length=17,
        quick_debug=True,
    )

    # データセットを作成
    dataset = MyEnvDataset(config)

    print(f"Dataset length: {len(dataset)}")

    # サンプルを取得
    sample = dataset[0]

    print(f"States shape: {sample.states.shape}")
    print(f"Actions shape: {sample.actions.shape}")
    if sample.locations is not None:
        print(f"Locations shape: {sample.locations.shape}")

    # 可視化
    if len(sample.states.shape) == 4: # 画像データ
        fig, axes = plt.subplots(1, 5, figsize=(15, 3))
        for i, ax in enumerate(axes):
            # 最初の5フレームを表示
            img = sample.states[i].permute(1, 2, 0).numpy()
            ax.imshow(img)
            ax.set_title(f"t={i}")
            ax.axis('off')
        plt.savefig('test_visualization.png')
        print("Saved visualization to test_visualization.png")
```

```
if __name__ == '__main__':  
    test_dataset()
```

8.2 統合テスト

```
# クイックデバッグモードで訓練を実行  
python pldm/train.py \  
    --configs pldm_envs/myenv/configs/default.yaml \  
    --values \  
        quick_debug=true \  
        epochs=1 \  
        data.myenv_config.crop_length=100
```

チェックリスト

新しい環境を追加する際の確認事項:

必須項目

- ☐ pldm_envs/myenv/ ディレクトリを作成
- ☐ pldm_envs/myenv/enums.py でSampleとConfigを定義
- ☐ pldm_envs/myenv/data/myenv_dataset.py でデータセットクラスを実装
- ☐ pldm/data/enums.py で DatasetType に追加
- ☐ pldm/data/enums.py で DataConfig に設定を追加
- ☐ pldm/data/dataset_factory.py でimportを追加
- ☐ pldm/data/dataset_factory.py で create_datasets() に分岐を追加
- ☐ pldm/data/dataset_factory.py で _create_myenv_datasets() を実装
- ☐ 設定ファイル pldm_envs/myenv/configs/default.yaml を作成
- ☐ データセットの動作確認テストを実行

オプション項目

- ☐ プロロービング用データセットの作成メソッドを実装
- ☐ pldm_envs/myenv/utils.py で PixelMapper を実装
- ☐ pldm/evaluation/evaluator.py の _create_pixel_mapper() に追加
- ☐ 評価用環境ジェネレータを実装
- ☐ プランニング設定を追加

- ☐ データ生成スクリプトを作成
- ☐ README.mdでデータセットの説明を記述

トラブルシューティング

よくあるエラー

1. ImportError: cannot import name 'MyEnvDatasetConfig'

- 原因: `pldm/data/enums.py` でimportを忘れている
- 解決: `from pldm_envs.myenv.enums import MyEnvDatasetConfig` を追加

2. KeyError: 'myenv_config' in DataConfig

- 原因: `DataConfig` に設定フィールドを追加していない
- 解決: `myenv_config: MyEnvDatasetConfig = MyEnvDatasetConfig()` を追加

3. AttributeError: 'MyEnvSample' object has no attribute 'states'

- 原因: `NamedTuple`のフィールド名が間違っている
- 解決: `states`, `actions` などの必須フィールドを確認

4. Normalizer関連のエラー

- 原因: `Normalizer`がサンプルの構造を理解できない
- 解決: `MyEnvSample` が標準的なフィールド名を使っているか確認

5. メモリ不足エラー

- 原因: 大容量データを一度にロードしている
- 解決: `mmap_mode='r'` を使用、または `lazy_load=True` を実装

参考実装

既存の実装を参考にする際のファイル:

シンプルな実装 (オンライン生成)

- [pldm_envs/wall/data/single.py](#) - DotDataset
- [pldm_envs/wall/data/wall.py](#) - WallDataset

オフラインデータセット

- [pldm_envs/wall/data/offline_wall.py](#) - OfflineWallDataset
- [pldm_envs/diverse_maze/d4rl.py](#) - D4RLDataset

DatasetFactory

- [pldm/data/dataset_factory.py](#) - すべてのデータセット作成ロジック

評価

- [pldm/evaluation/evaluator.py](#) - Evaluatorの実装
- [pldm_envs/diverse_maze/utils.py](#) - PixelMapperの例

まとめ

新しい環境を追加するには:

1. 環境ディレクトリとデータセットクラスを作成 ([pldm_envs/myenv/](#))
2. 列挙型と設定を更新 ([pldm/data/enums.py](#))
3. **DatasetFactory**に作成ロジックを追加 ([pldm/data/dataset_factory.py](#))
4. 設定ファイルを作成 ([pldm_envs/myenv/configs/](#))
5. (オプション) 評価ロジックを追加 ([pldm/evaluation/evaluator.py](#))

最小限の実装であれば、ステップ1-4のみで訓練を開始できます。

プロービングやプランニングによる評価が必要な場合は、ステップ5も実装してください。